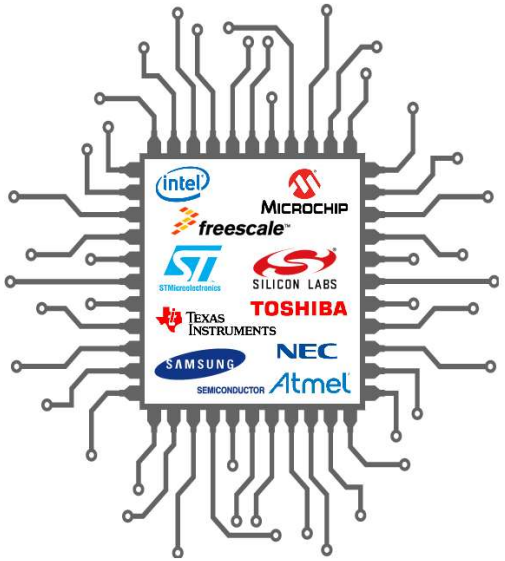
Polling and Interrupt Handling



Thareendhra Keerthi Wijayasiriwardhane

Department of Industrial Management
Faculty of Science
University of Kelaniya
thareen@kln.ac.lk

Learning Outcomes

- On completion of this session, you should be able to
  - Compare and contrast polling vs. interrupts
  - Construct a microcontroller-based system and program it to preform more than one tasks seemingly at the same time using polling
  - Explain how interrupt handling works
  - Explain the functionality of interrupt registers
  - Construct a microcontroller-based system and program it to perform more than one task seemingly at the same time using interrupts

Polling and Interrupt

- Two approaches to preform more than one tasks seemingly at the same time
- Used to provide CPU's service to the peripheral devices when CPU is busy with another task

Polling

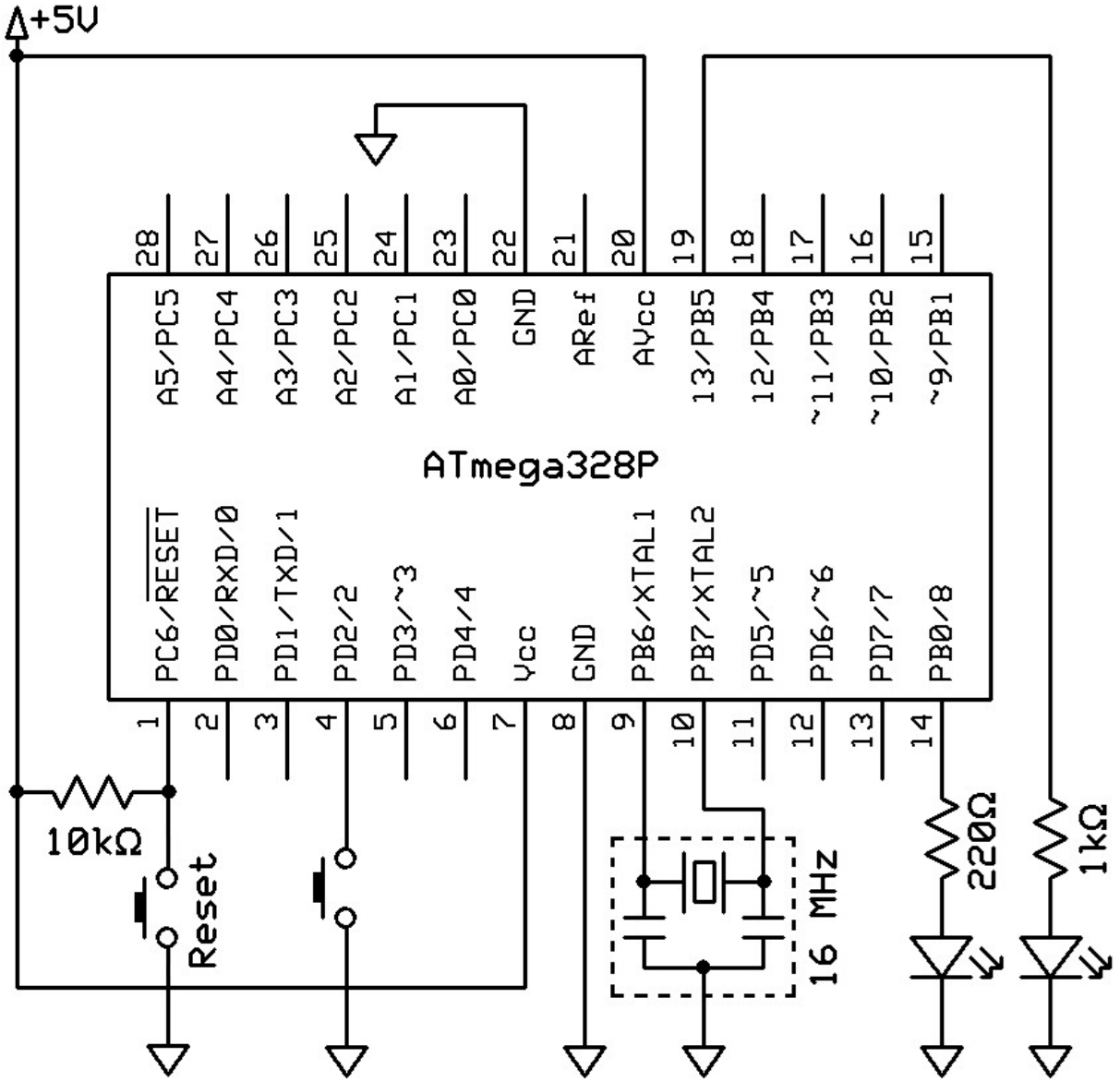
- CPU keeps on checking status of peripheral devices at regular intervals repeatedly while performing another task
- When the status condition of a device is met, CPU provides its service to the device and then resumes the task it was doing
- CPU then starts again monitoring the status of devices

System 5

pollingBasedTrigger - Toggles an LED on user input while blinking another LED in every 2 seconds using polling

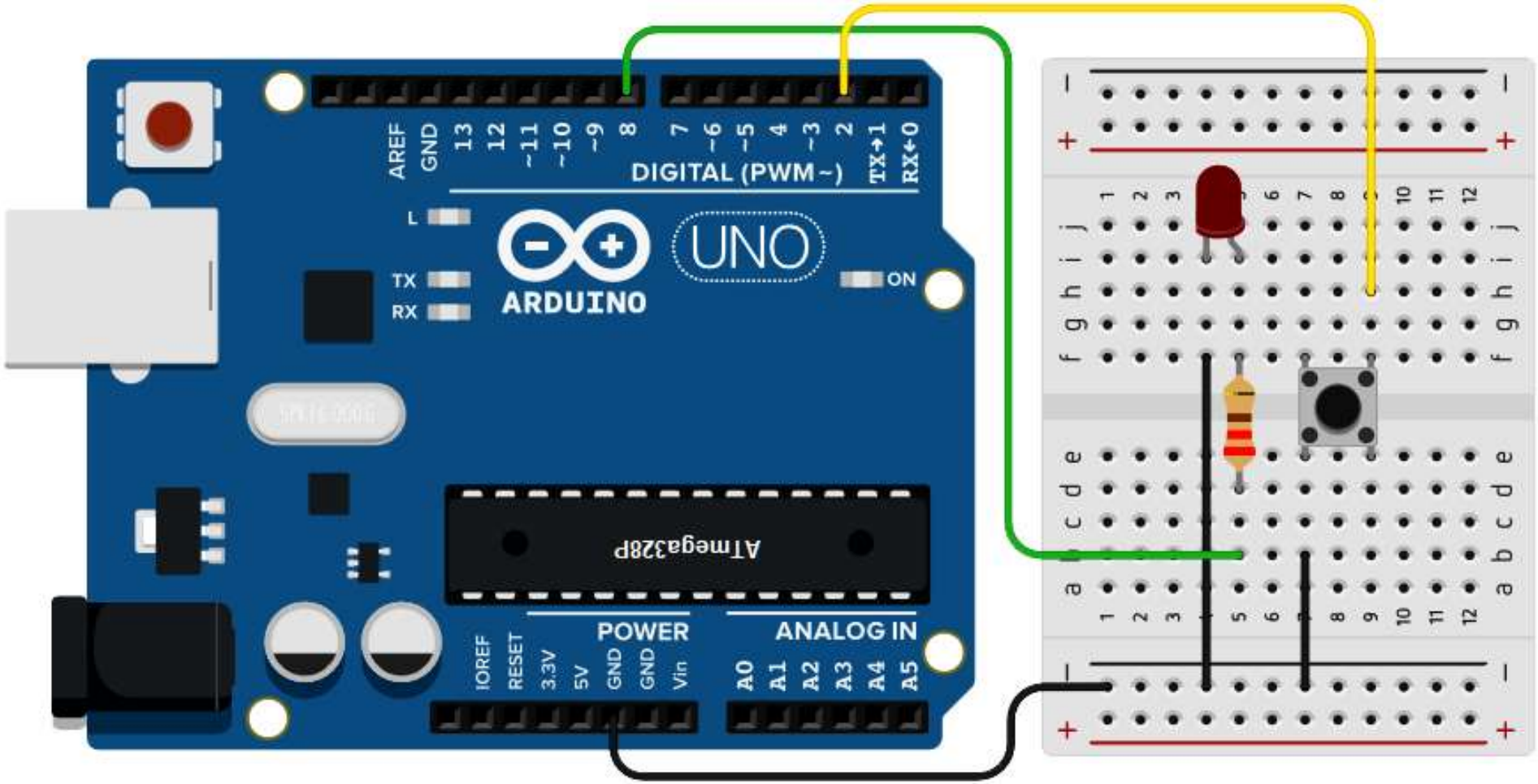
Hardware

- Schematic Diagram



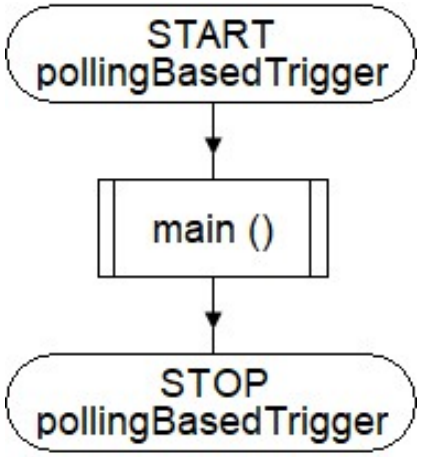
Hardware

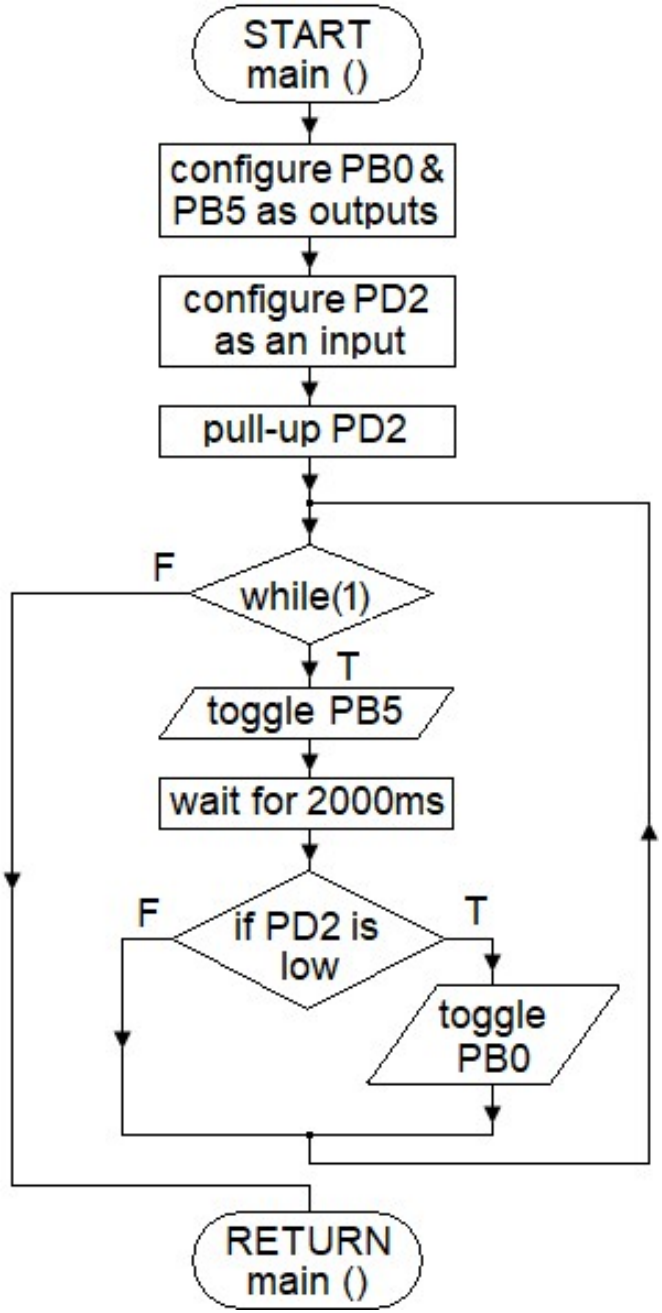
- Wiring Diagram



Firmware

- Algorithm



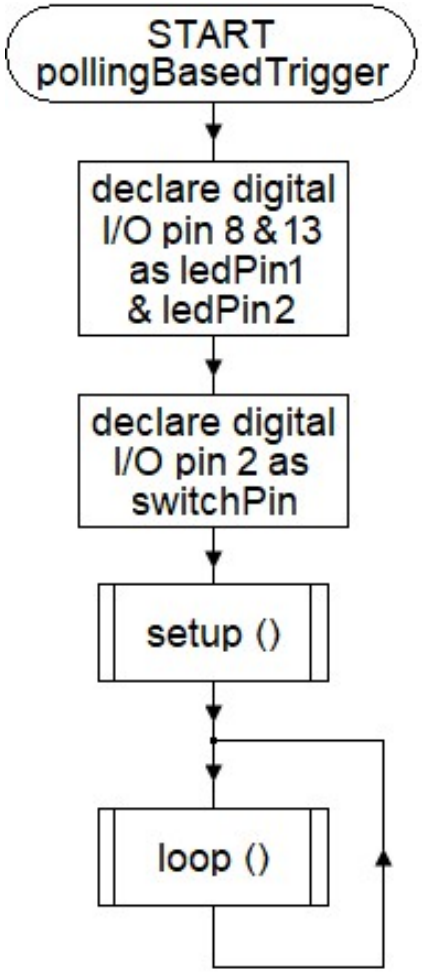


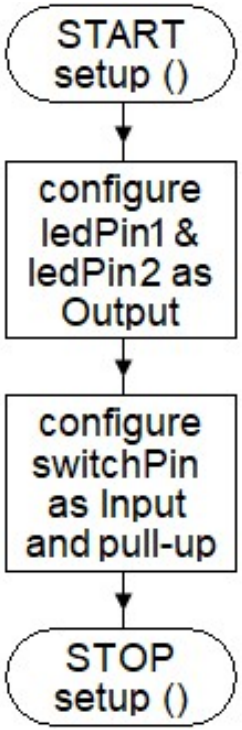
C Program

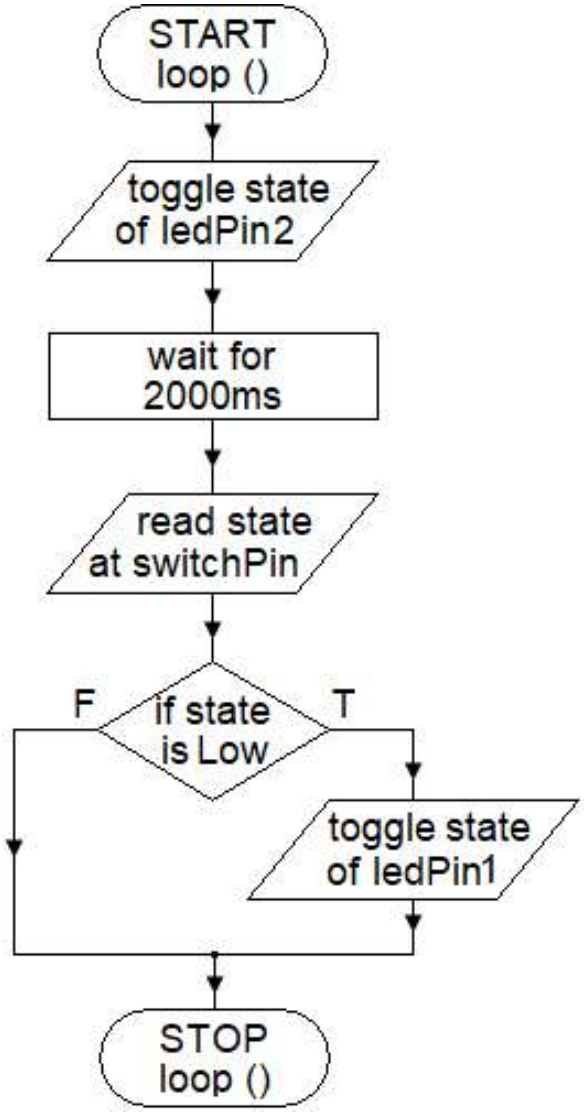
```
1   // pollingBasedTriggerC.c
2
3   #include <avr/io.h>
4   #define F_CPU 16000000UL                    // define CPU clock speed for delay.h
5   #include <util/delay.h>
6
7   int main(void)
8   {
9       DDRB = DDRB | 0b00100001;               // configure PB0 & PB5 as outputs
10      DDRD = DDRD & 0b11111011;               // configure PD2 as an input
11      PORTD = PORTD | 0b00000100;             // pull-up PD2
12      while (1)
13      {
14          PORTB = PORTB ^ 0b00100000;         // toggle state of PB5
15          _delay_ms(2000);                    // delay 2000ms
16          if ((PIND & 0b00000100) == 0)       // if state of PD2 is low
17              PORTB = PORTB ^ 0b00000001;     // toggle state of PB0
18      }
19      return 0;
20  }
```

Firmware

- Algorithm for Arduino







Firmware

- Source Code (using Arduino Wrapper Functions)

```
1 // pollingBasedTrigger.ino
2
3 const int ledPin1 = 8;                 // declare digital I/O pin 8 as ledPin1
4 const int ledPin2 = 13;                // declare digital I/O pin 13 as ledPin2
5 const int switchPin = 2;               // declare digital I/O pin 2 as switchPin
6
7 void setup()
8 {
9   pinMode(ledPin1, OUTPUT);            // configure ledPin1 as output
10  pinMode(ledPin2, OUTPUT);            // configure ledPin2 as output
11  pinMode(switchPin, INPUT_PULLUP); // configure switchPin as input and pull-up
12 }
13
14 void loop()
15 {
16  digitalWrite(ledPin2, !digitalRead(ledPin2));   // toggle state of ledPin2
17  delay(2000);                         // wait for 2000ms
18  if (!digitalRead(switchPin))         // read state at switchPin and if it is low
19    digitalWrite(ledPin1, !digitalRead(ledPin1)); // toggle state of ledPin1
20 }
```

Polling

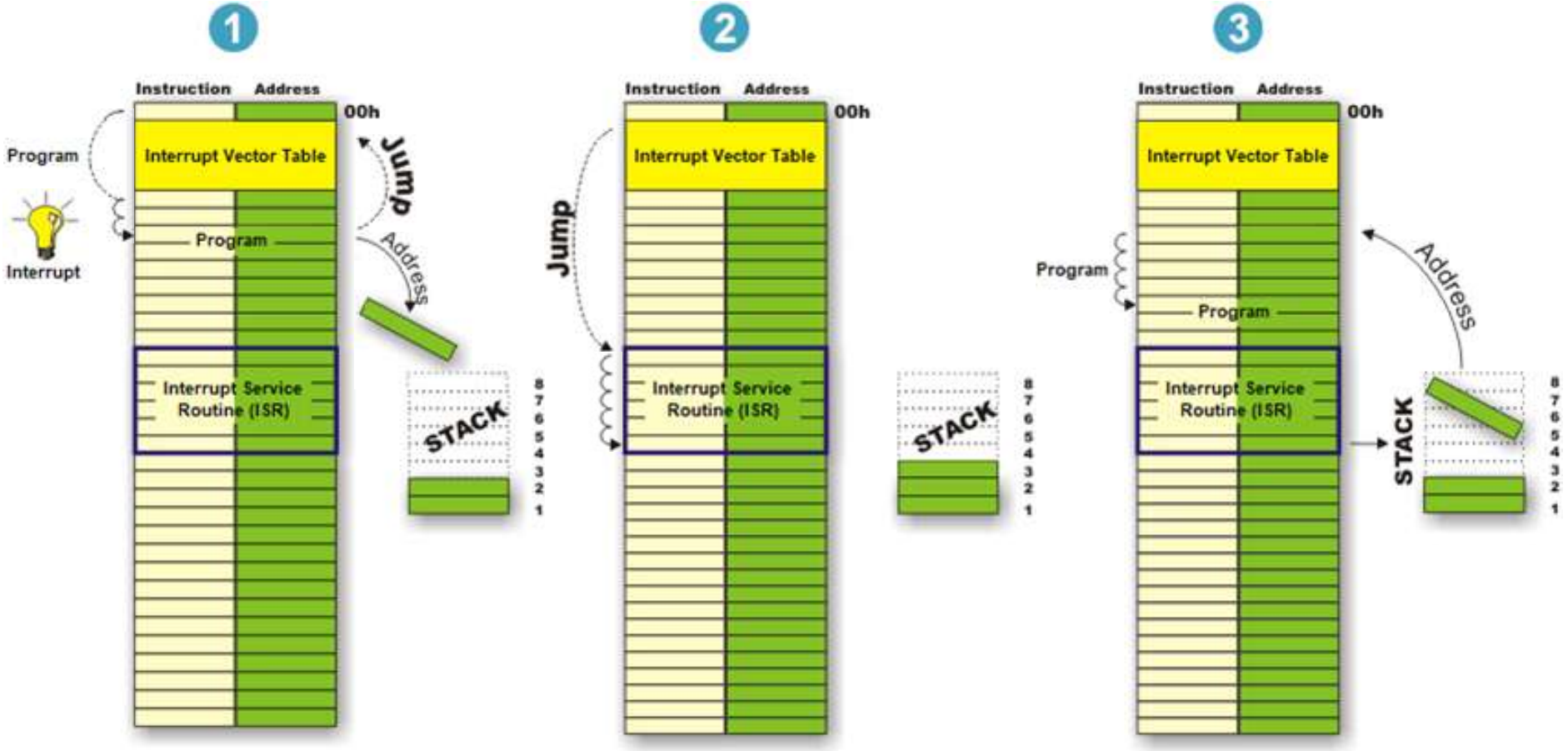
- Problems
  - If the state of a device changes too fast or the CPU's device monitoring interval is too long, the state change of the device may go undetected
  - No support for the concept of priority as CPU repeatedly performs all tasks in the same order

Interrupt

- Peripheral devices interrupt the CPU when they need CPU's service while CPU is busy with another task
- CPU suspends the currently performing task and provides its service to the interrupted device
- CPU then resumes the task it was doing
- Frees CPU from monitoring (polling) the state of peripheral devices
- Supports the concept of priority

Interrupt Handling

- Steps



Interrupt Vector Table

- Widely used Interrupt Vectors

Address	Interrupt	Interrupt_Vector
0x0002	External interrupt on INT0 pin	INT0_vect
0x0004	External interrupt on INT1 pin	INT1_vect
0x0006	Pin change interrupt on Port B pins	PCINT0_vect
0x0008	Pin change interrupt on Port C pins	PCINT1_vect
0x000A	Pin change interrupt on Port D pins	PCINT2_vect
0x0012	Timer/Counter 2 overflow interrupt	TIMER2_OVF_vect
0x001A	Timer/Counter 1 overflow interrupt	TIMER1_OVF_vect
0x0020	Timer/Counter 0 overflow interrupt	TIMER0_OVF_vect
0x0024	USART receive complete interrupt	USART_RX_vect
0x0028	USART transmit complete interrupt	USART_TX_vect
0x002A	ADC conversion complete interrupt	ADC_vect

Interrupt Routines

- Interrupt Service Routine

```
ISR (Interrupt_Vector)
{

    // code for handling interrupt

}
```

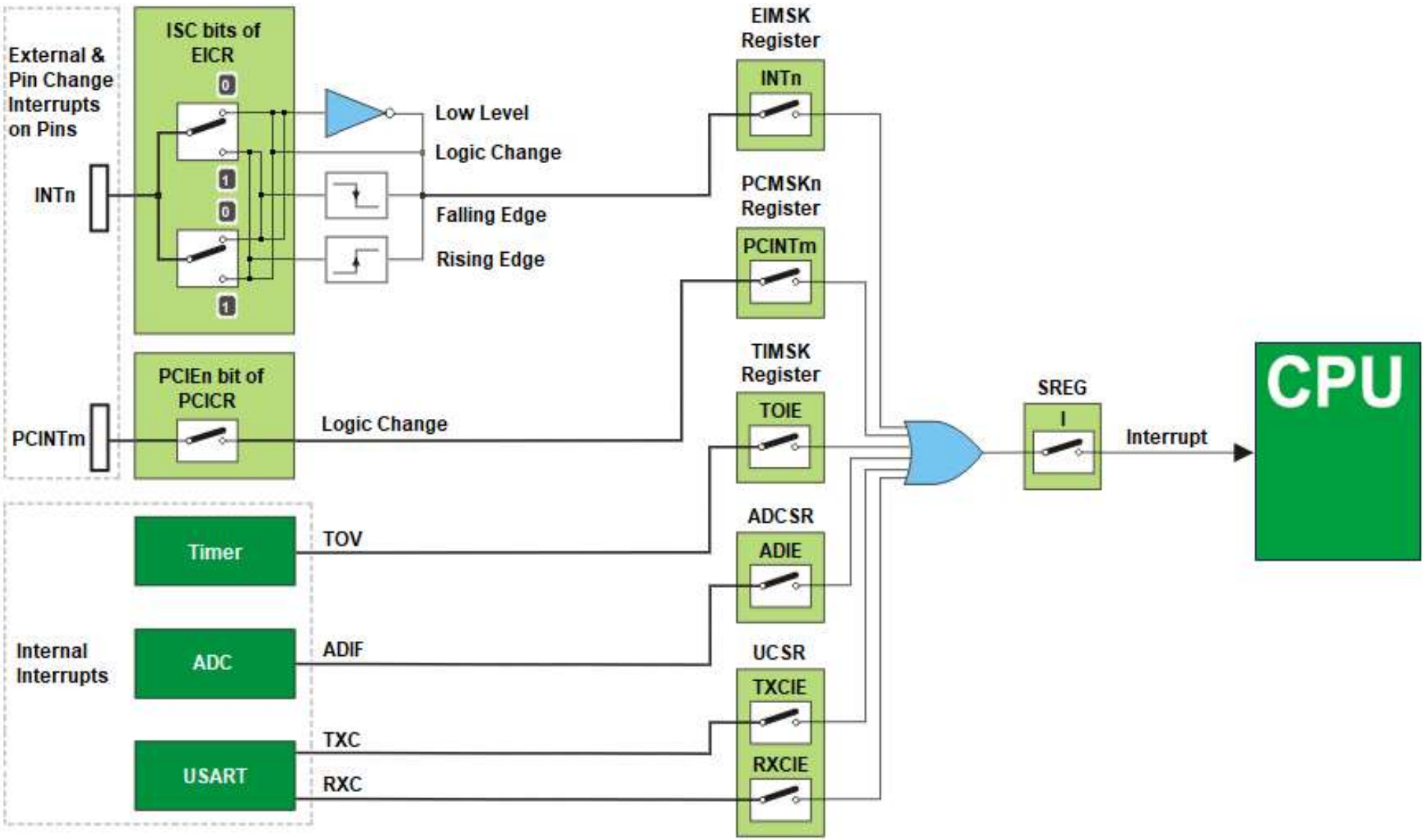
- SEt Interrupt

```
sei()
```

- CLear Interrupt

```
cli()
```

Interrupt Registers



External Interrupt Registers

- EICRA (External Interrupt Control Register A)



  - ISC01 (Interrupt Sense Control 01) and ISC00 bits specify when to interrupt CPU on INT0 pin

ISC01	ISC00	Interrupt on
0	0	Low level of INT0 pin
0	1	Logic change of INT0 pin
1	0	Falling edge of INT0 pin
1	1	Rising edge of INT0 pin

External Interrupt Registers

- EICRA (External Interrupt Control Register A)



  - ISC11 (Interrupt Sense Control 11) and ISC10 bits specify when to interrupt CPU on INT1 pin

ISC11	ISC10	Interrupt on
0	0	Low level of INT1 pin
0	1	Logic change of INT1 pin
1	0	Falling edge of INT1 pin
1	1	Rising edge of INT1 pin

External Interrupt Registers

- EIMSK (External Interrupt MaSK) register

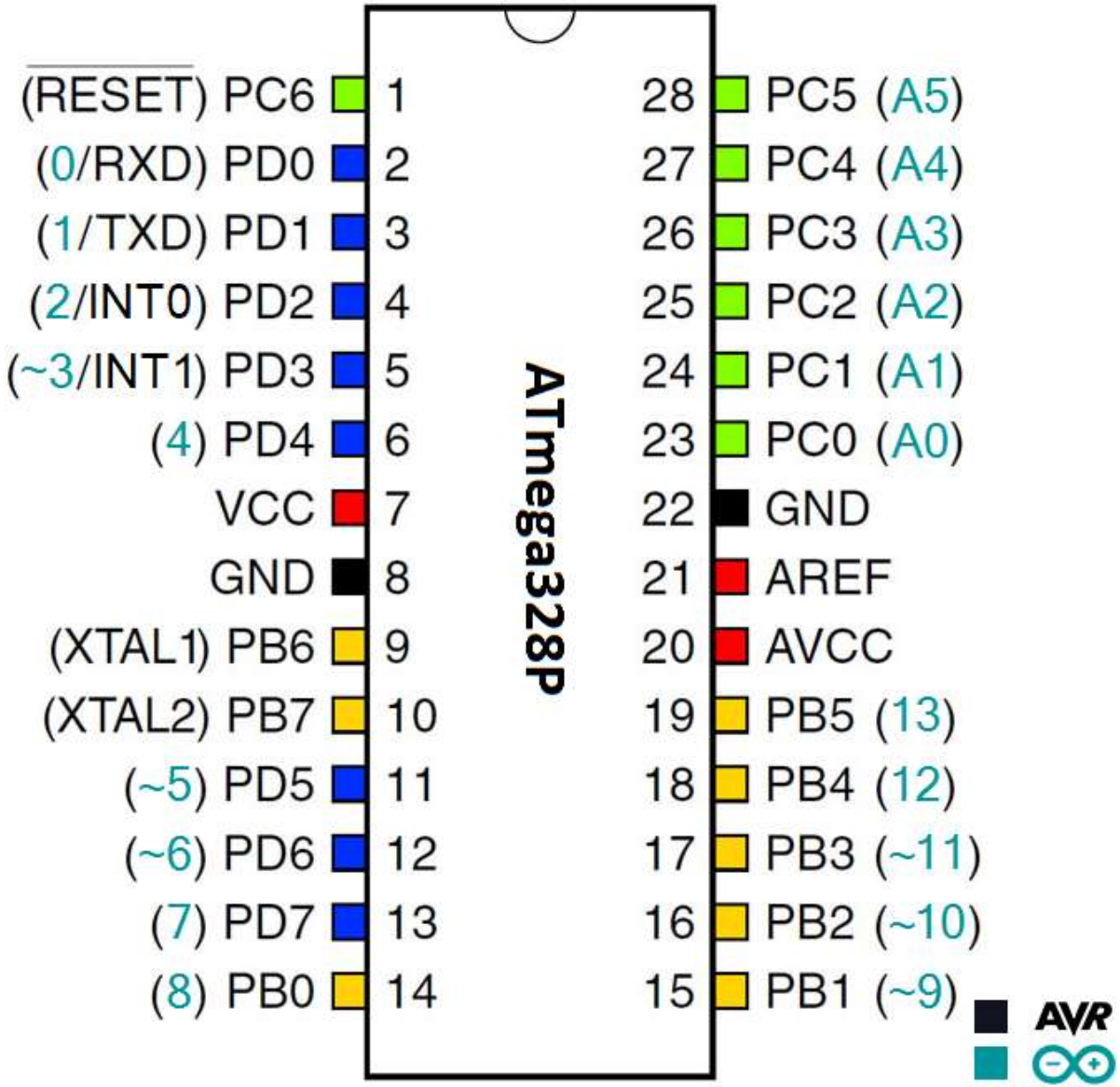


  - INTn (INTerrupt n) bit enables (or disables) interrupts on INTn pin

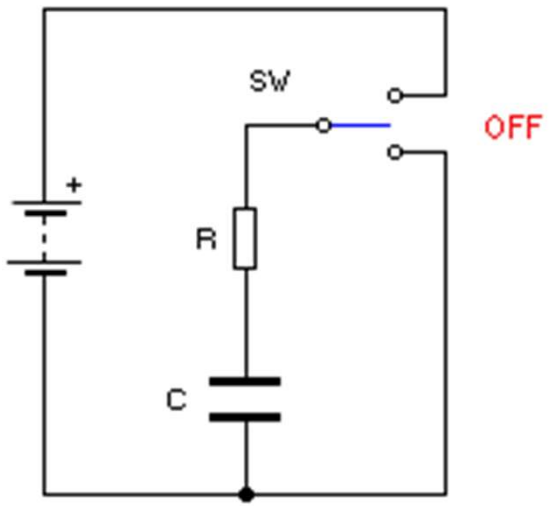
System 6

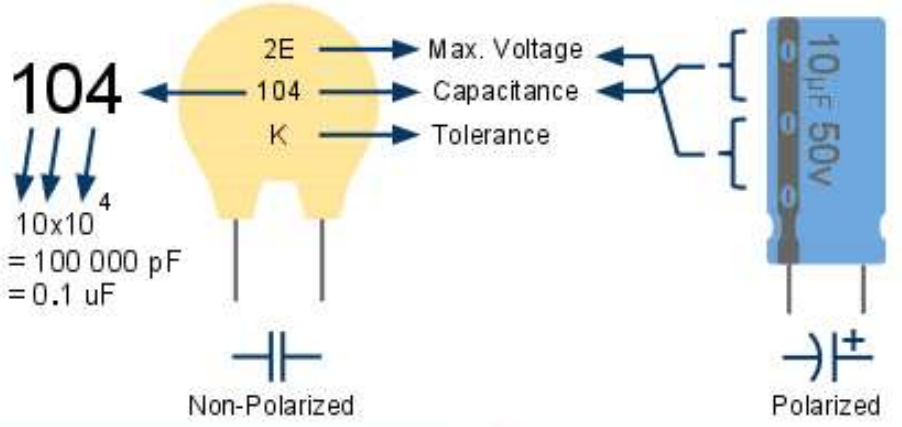
externalInterruptBasedTrigger - Toggles an LED on user input while blinking another LED in every 2 seconds using external interrupts

ATmega328P External Interrupt Pinout



Capacitor



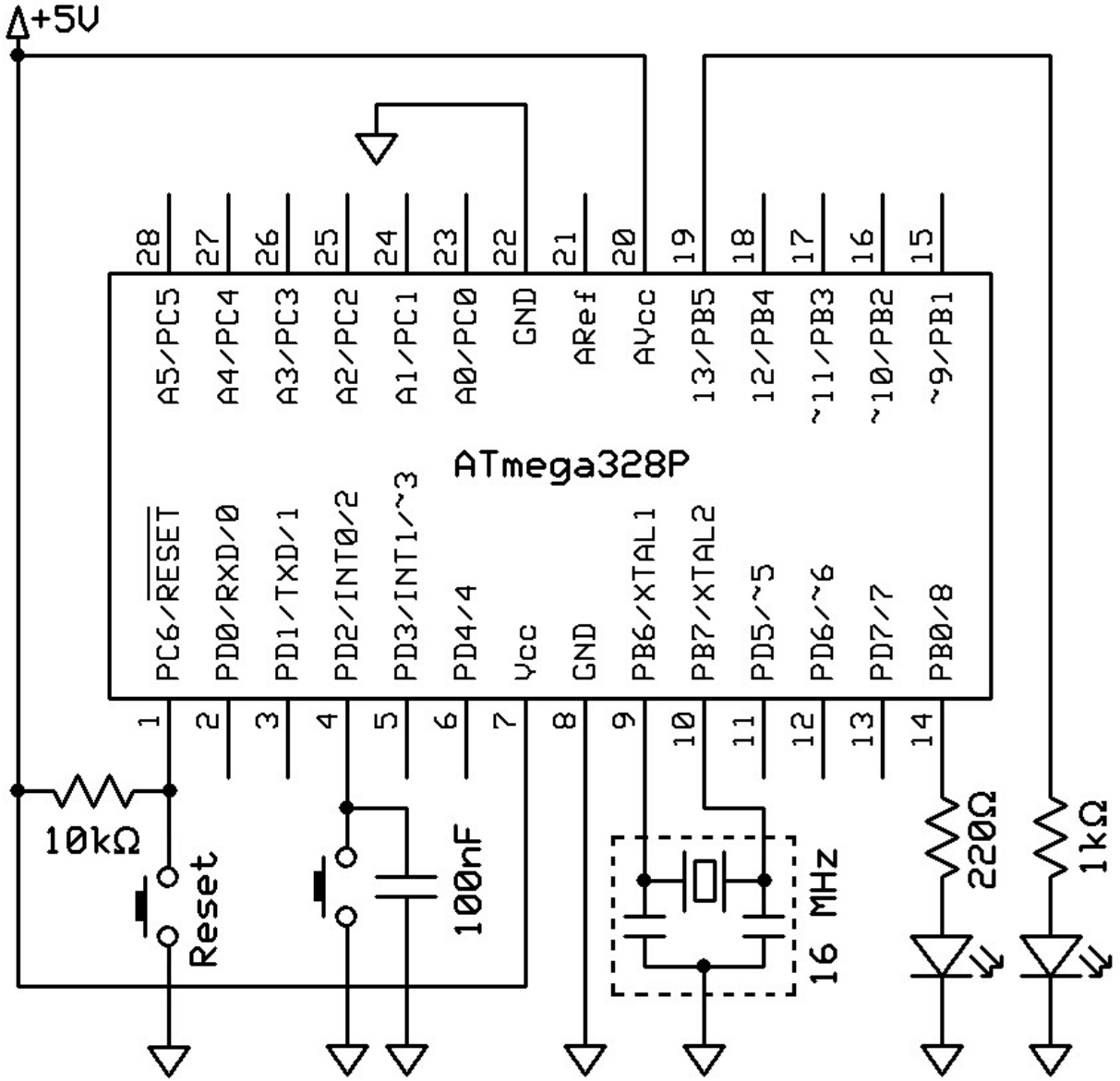


Max. Operating Voltage	
Code	Max. Voltage
1H	50V
2A	100V
2T	150V
2D	200V
2E	250V
2G	400V
2J	630V

Tolerance	
Code	Percentage
B	± 0.1 pF
C	±0.25 pF
D	±0.5 pF
F	±1%
G	±2%
H	±3%
J	±5%
K	±10%
M	±20%

Hardware

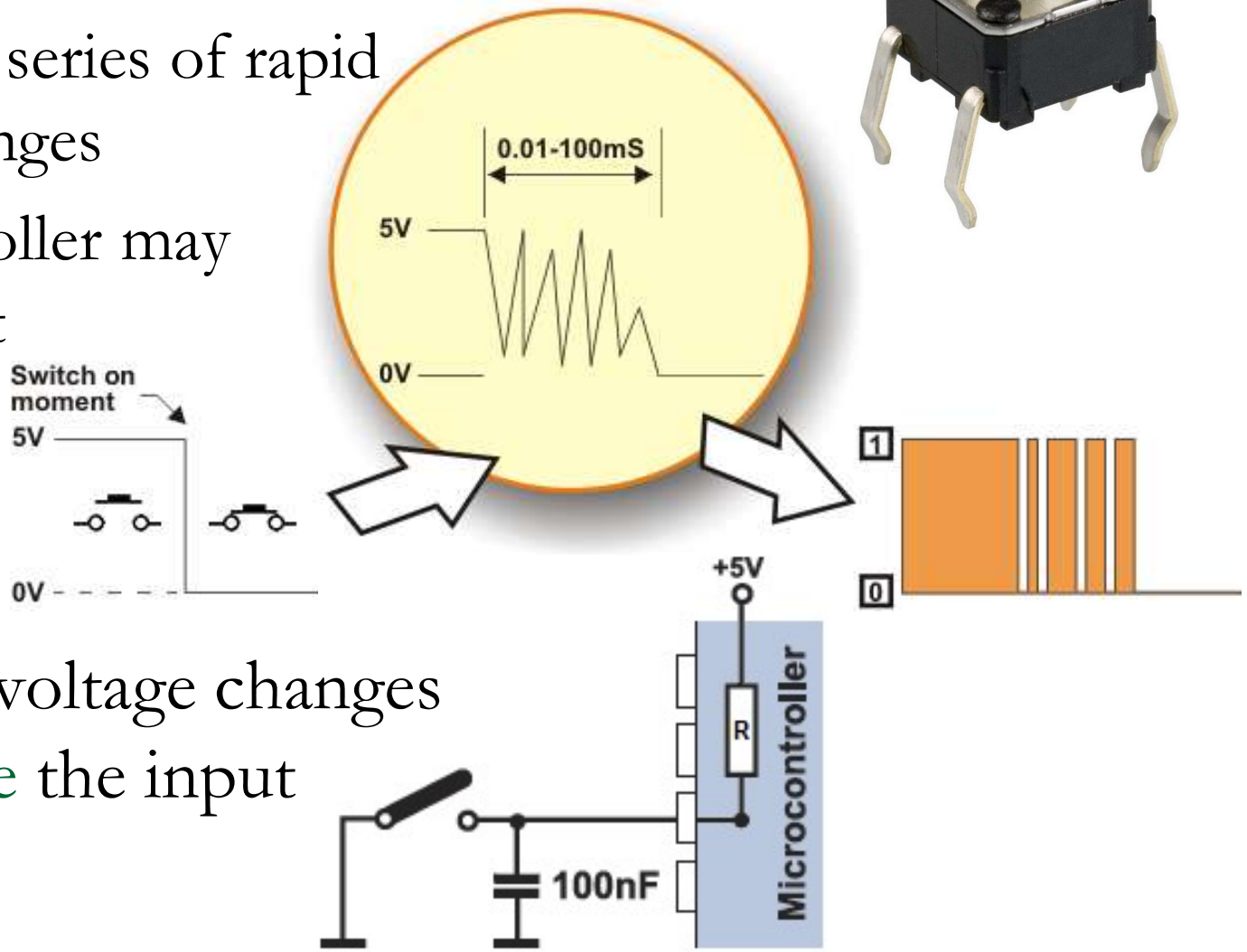
- Schematic Diagram



Debouncing Inputs

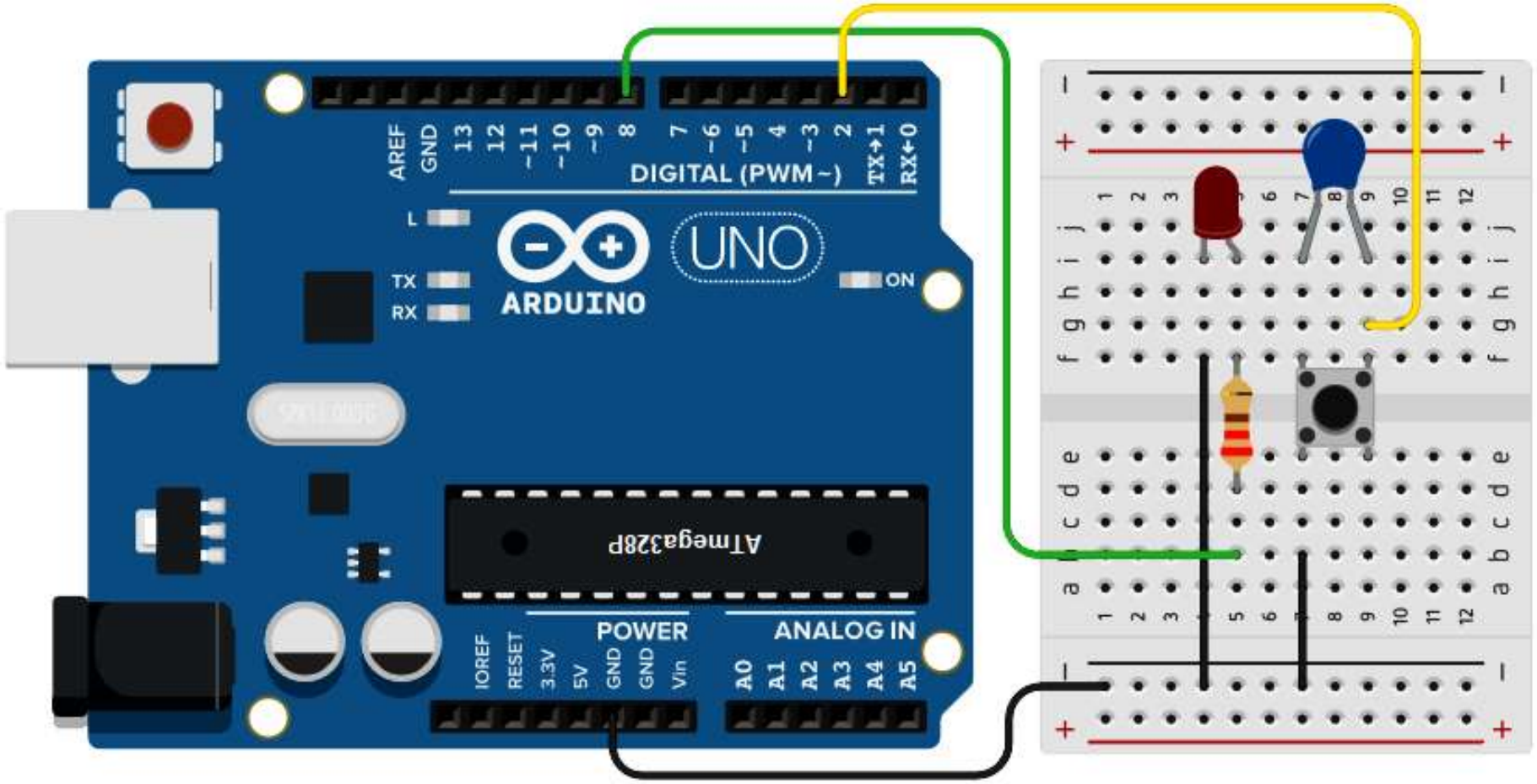
- Contacts bounce on mechanical switches
  - Results in a series of rapid voltage changes
  - Microcontroller may misinterpret the input
- Use a capacitor to suppress the voltage changes and debounce the input





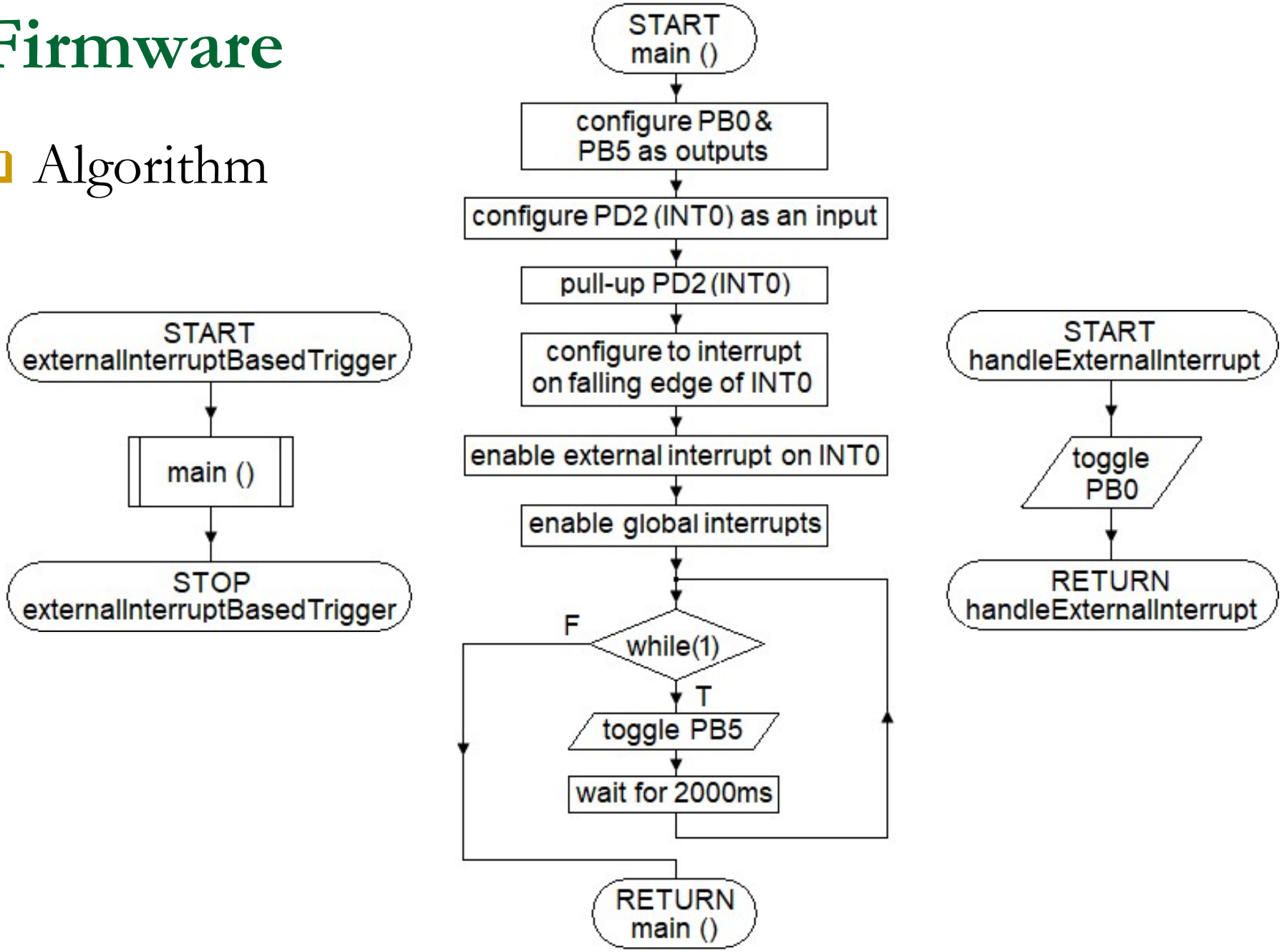
Hardware

- Wiring Diagram



Firmware

- Algorithm

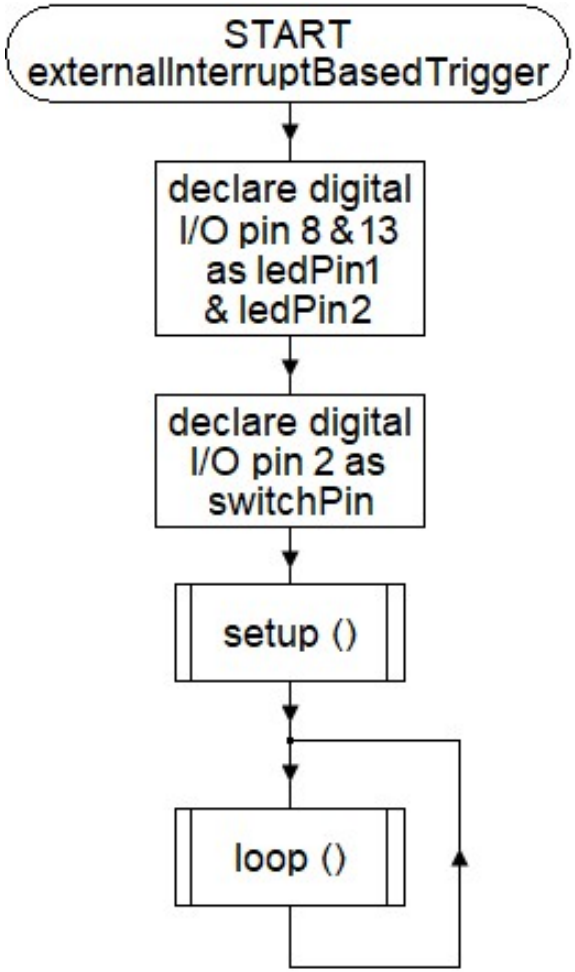


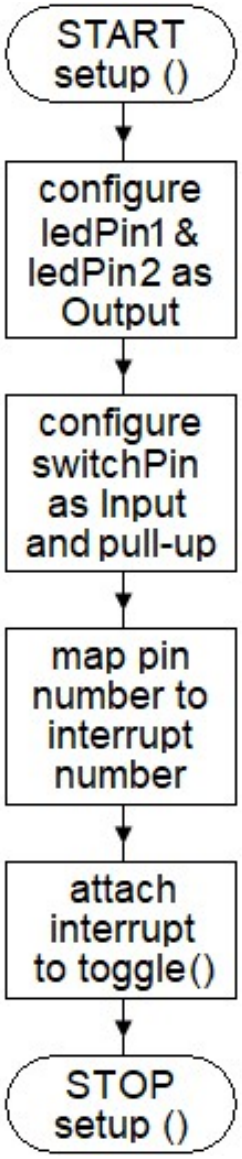
C Program

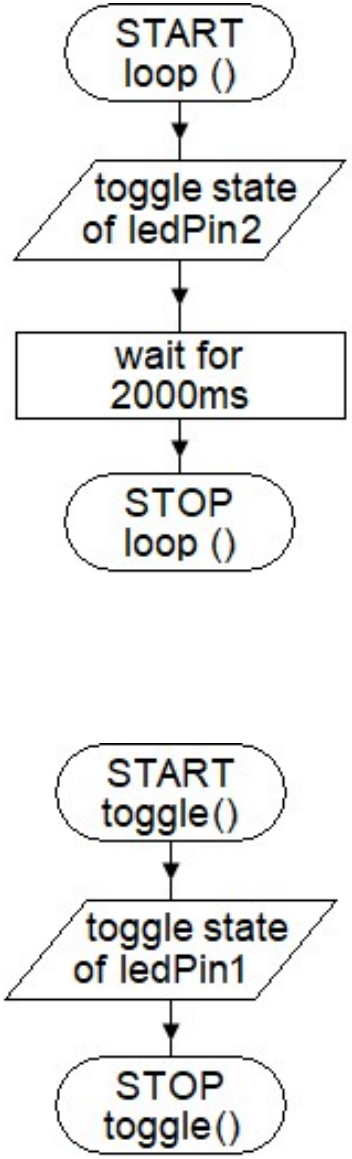
```
1   // externalInterruptBasedTriggerC.c
2
3   #include <avr/io.h>
4   #define F_CPU 16000000UL                        // define CPU clock speed for delay.h
5   #include <util/delay.h>
6   #include <avr/interrupt.h>
7
8   int main(void)
9   {
10      DDRB = DDRB | 0b00100001;                   // configure PB5 and PB0 of Port B as outputs
11      DDRD = DDRD & 0b11111011;                   // configure PD2 (INT0) of Port D as an input
12      PORTD = PORTD | 0b00000100;                 // pull-up PD2 (INT0) of Port D
13      EICRA = (EICRA & 0b11111110) | 0b00000010; // configure to interrupt on falling edge of INT0
14      EIMSK = EIMSK | 0b00000001;                 // enable external interrupt on INT0
15      sei();                                      // enable global interrupts
16      while(1)
17      {
18          PORTB = PORTB ^ 0b00100000;             // toggle state of PB5 of Port B
19          _delay_ms(2000);                        // delay 2000ms
20      }
21      return 0;
22  }
23
24  ISR(INT0_vect)
25  {
26      PORTB = PORTB ^ 0b00000001;                 // toggle state of PB0 of Port B
27  }
```

Firmware

- Algorithm for Arduino







Firmware

- Source Code (using Arduino Wrapper Functions)

```
1 // externalInterruptBasedTrigger.ino
2 const int ledPin1 = 8;                          // declare I/O pin 8 as ledPin1
3 const int ledPin2 = 13;                         // declare I/O pin 13 as ledPin2
4 const int switchPin = 2;                        // declare I/O pin 2 as switchPin
5 void setup()
6 {
7   pinMode(ledPin1, OUTPUT);                     // configure ledPin1 as output
8   pinMode(ledPin2, OUTPUT);                     // configure ledPin2 as output
9   pinMode(switchPin, INPUT_PULLUP);             // configure switchPin as input & pull-up
10  int intNum = digitalPinToInterrupt(switchPin);// map I/O pin to interrupt number
11  attachInterrupt(intNum, toggle, FALLING);     // attach inetrrupt to toggle()
12 }
13 void loop()
14 {
15  digitalWrite(ledPin2, !digitalRead(ledPin2)); // toggle state of ledPin2
16  delay(2000);                                  // wait for 2000ms
17 }
18 void toggle()                                   // interrupt service routine
19 {
20  digitalWrite(ledPin1, !digitalRead(ledPin1)); // toggle state of ledPin1
21 }
```

Pin Change Interrupt Registers

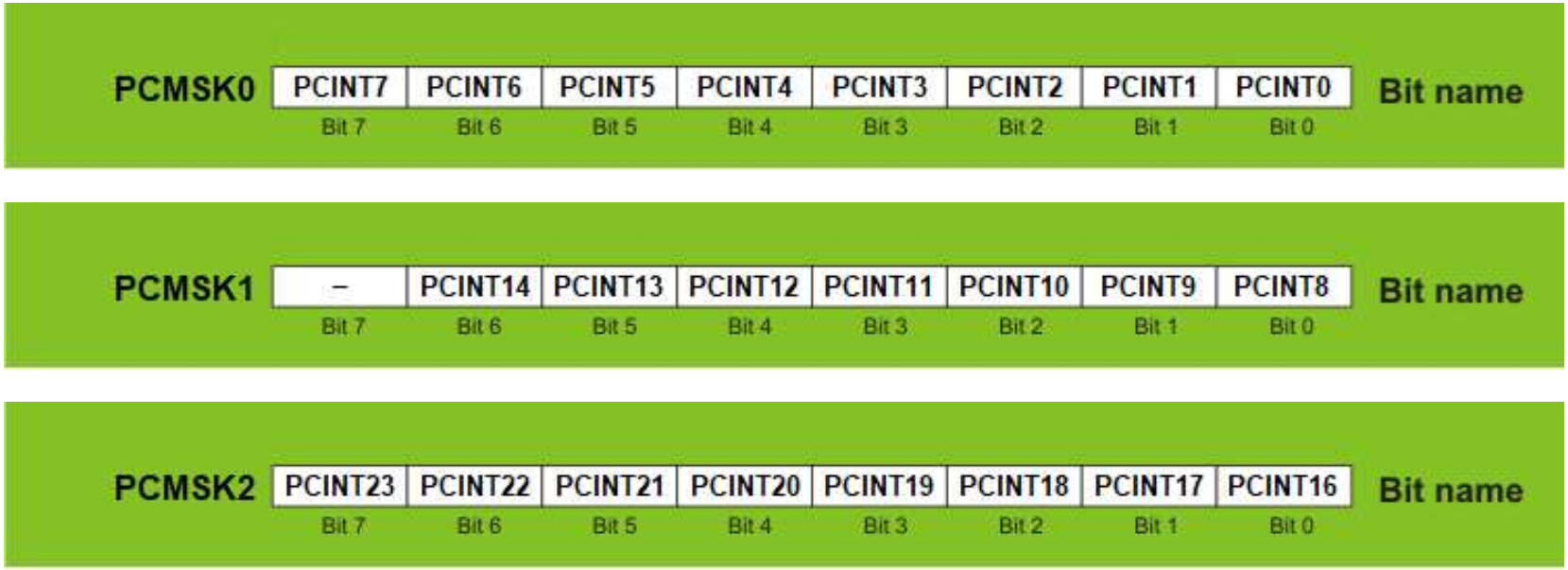
- PCICR (Pin Change Interrupt Control Register)



  - PCIE2 (Pin Change Interrupt Enable 2), PCIE1 and PCIE0 bits enable interrupts on Port D, Port C and Port B, respectively

Pin Change Interrupt Registers

- PCMSKn (Pin Change MaSK n) register



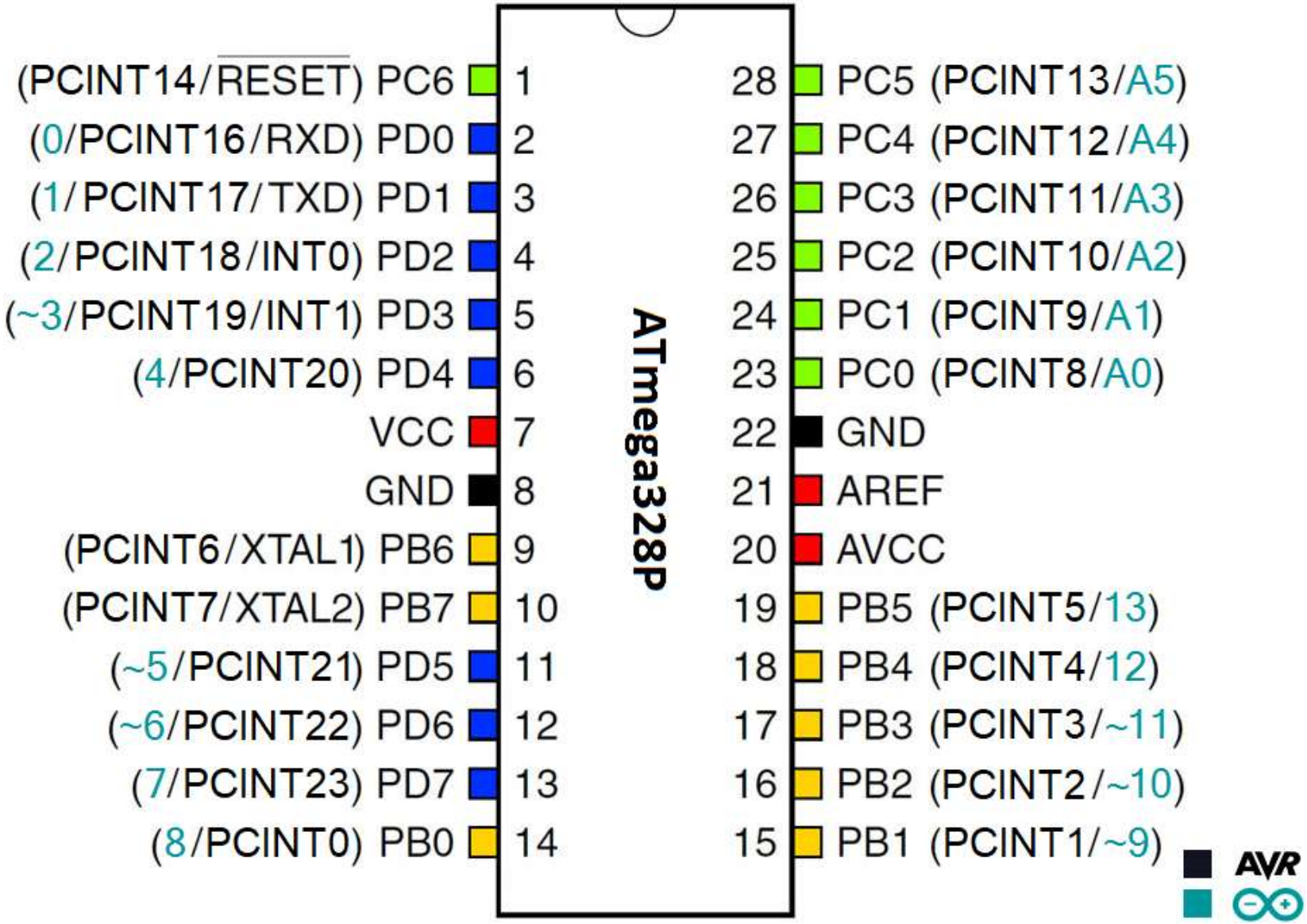
PCMSK0	PCINT7	PCINT6	PCINT5	PCINT4	PCINT3	PCINT2	PCINT1	PCINT0	Bit name
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	
PCMSK1	–	PCINT14	PCINT13	PCINT12	PCINT11	PCINT10	PCINT9	PCINT8	Bit name
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	
PCMSK2	PCINT23	PCINT22	PCINT21	PCINT20	PCINT19	PCINT18	PCINT17	PCINT16	Bit name
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	

  - PCINTm (Pin Change INTerrupt m) bits enable (or disable) interrupts on pins of an enabled Port

System 7

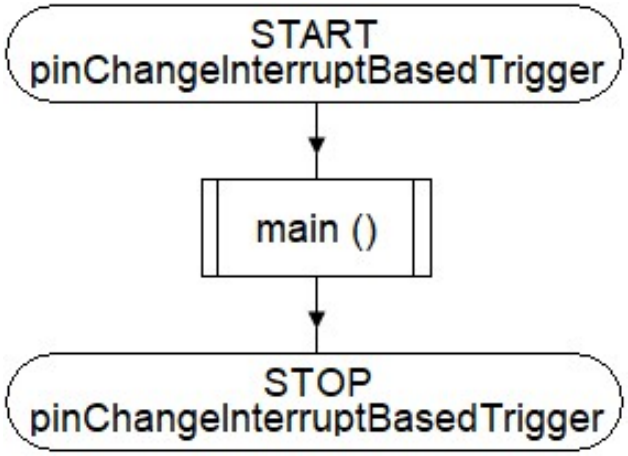
pinChangeInterruptBasedTrigger - Toggles an LED on user input while blinking another LED in every 2 seconds using pin change interrupts

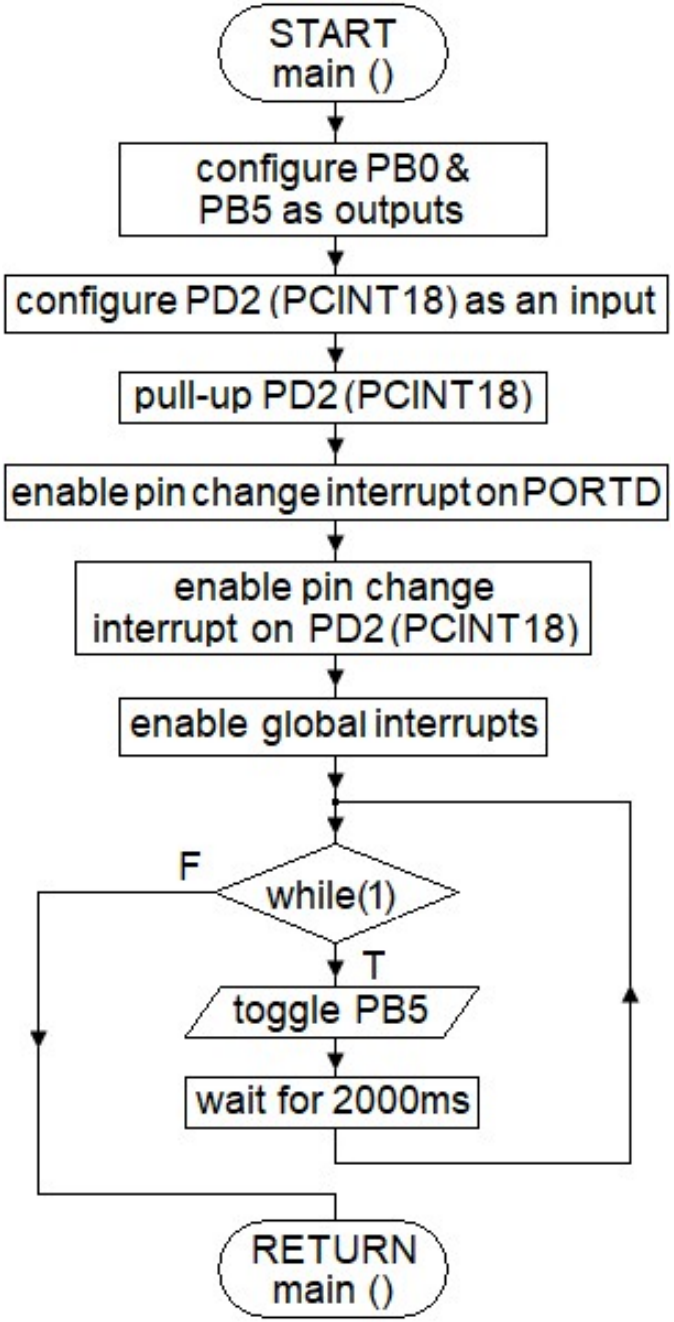
ATmega328P Interrupt Pinout

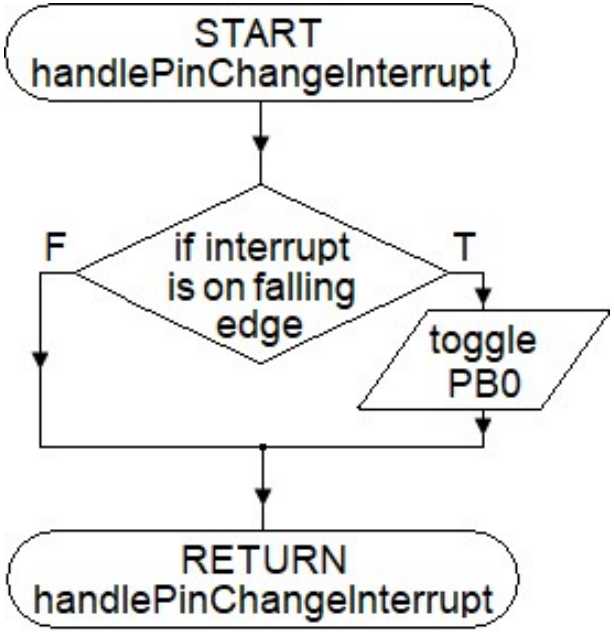


Firmware

- Algorithm







C Program

```
1  // pinChangeInterruptBasedTriggerC.c
2  #include <avr/io.h>
3  #define F_CPU 16000000UL                    // define CPU clock speed for delay.h
4  #include <util/delay.h>
5  #include <avr/interrupt.h>
6
7  int main(void)
8  {
9      DDRB = DDRB | 0b00100001;               // configure PB5 and PB0 of Port B as outputs
10     DDRD = DDRD & 0b11111011;               // configure PD2 (PCINT18) of Port D as an input
11     PORTD = PORTD | 0b00000100;             // pull-up PD2 (PCINT18) of Port D
12     PCICR = PCICR | 0b00000100;             // enable pin change interrupt on Port D
13     PCMSK2 = PCMSK2 | 0b00000100;           // enable pin change interrupt on PD2 (PCINT18)
14     sei();                                  // enable global interrupts
15     while(1)
16     {
17         PORTB = PORTB ^ 0b00100000;         // toggle state of PB5 of Port B
18         _delay_ms(2000);                    // delay 2000ms
19     }
20     return 0;
21 }
22
23 ISR(PCINT2_vect)                           // handle Pin Change Interrupts on Port D pins
24 {
25     if ((PIND & 0b00000100) == 0)           // if state of PD2 (PCINT18) is low
26         PORTB = PORTB ^ 0b00000001;         // toggle state of PB0 of Port B
27 }
```

Using Hexadecimal Constants

```
1  // pinChangeInterruptBasedTriggerC.c
2  #include <avr/io.h>
3  #define F_CPU 16000000UL                    // define CPU clock speed for delay.h
4  #include <util/delay.h>
5  #include <avr/interrupt.h>
6
7  int main(void)
8  {
9      DDRB = DDRB | 0x21;                     // configure PB5 and PB0 of Port B as outputs
10     DDRD = DDRD & 0xFB;                     // configure PD2 (PCINT18) of Port D as an input
11     PORTD = PORTD | 0x04;                   // pull-up PD2 (PCINT18) of Port D
12     PCICR = PCICR | 0x04;                   // enable pin change interrupt on Port D
13     PCMSK2 = PCMSK2 | 0x04;                 // enable pin change interrupt on PD2 (PCINT18)
14     sei();                                  // enable global interrupts
15     while(1)
16     {
17         PORTB = PORTB ^ 0x20;               // toggle state of PB5 of Port B
18         _delay_ms(2000);                    // delay 2000ms
19     }
20     return 0;
21 }
22
23 ISR(PCINT2_vect)                           // handle Pin Change Interrupts on Port D pins
24 {
25     if ((PIND & 0x04) == 0)                 // if state of PD2 (PCINT18) is low
26         PORTB = PORTB ^ 0x01;               // toggle state of PB0 of Port B
27 }
```

Using Bit-wise Shift Operators

```
1  // pinChangeInterruptBasedTriggerC.c
2  #include <avr/io.h>
3  #define F_CPU 16000000UL                    // define CPU clock speed for delay.h
4  #include <util/delay.h>
5  #include <avr/interrupt.h>
6
7  int main(void)
8  {
9      DDRB = DDRB | (1<<5) | (1<<0);          // configure PB5 and PB0 of Port B as outputs
10     DDRD = DDRD & ~(1<<2);                  // configure PD2 (PCINT18) of Port D as an input
11     PORTD = PORTD | (1<<2);                 // pull-up PD2 (PCINT18) of Port D
12     PCICR = PCICR | (1<<2);                 // enable pin change interrupt on Port D
13     PCMSK2 = PCMSK2 | (1<<2);               // enable pin change interrupt on PD2 (PCINT18)
14     sei();                                  // enable global interrupts
15     while(1)
16     {
17         PORTB = PORTB ^ (1<<5);             // toggle state of PB5 of Port B
18         _delay_ms(2000);                    // delay 2000ms
19     }
20     return 0;
21 }
22
23 ISR(PCINT2_vect)                           // handle Pin Change Interrupts on Port D pins
24 {
25     if ((PIND & (1<<2)) == 0)               // if state of PD2 (PCINT18) is low
26         PORTB = PORTB ^ (1<<0);             // toggle state of PB0 of Port B
27 }
```

Using Labeled Constants

```
1  // pinChangeInterruptBasedTriggerC.c
2  #include <avr/io.h>
3  #define F_CPU 16000000UL                    // define CPU clock speed for delay.h
4  #include <util/delay.h>
5  #include <avr/interrupt.h>
6
7  int main(void)
8  {
9      DDRB = DDRB | (1<<DDB5) | (1<<DDB0);    // configure PB5 and PB0 of Port B as outputs
10     DDRD = DDRD & ~(1<<DDD2);               // configure PD2 (PCINT18) of Port D as an input
11     PORTD = PORTD | (1<<PORTD2);            // pull-up PD2 (PCINT18) of Port D
12     PCICR = PCICR | (1<<PCIE2);             // enable pin change interrupt on Port D
13     PCMSK2 = PCMSK2 | (1<<PCINT18);         // enable pin change interrupt on PD2 (PCINT18)
14     sei();                                  // enable global interrupts
15     while(1)
16     {
17         PORTB = PORTB ^ (1<<PORTB5);        // toggle state of PB5 of Port B
18         _delay_ms(2000);                    // delay 2000ms
19     }
20     return 0;
21 }
22
23 ISR(PCINT2_vect)                           // handle Pin Change Interrupts on Port D pins
24 {
25     if ((PIND & (1<<PIND2)) == 0)           // if state of PD2 (PCINT18) is low
26         PORTB = PORTB ^ (1<<PORTB0);        // toggle state of PB0 of Port B
27 }
```

Questions